Deployment Guide

ECITY — Mobile Shop Management Web App

Version 1.2 · 2026

Buying the server, deploying, backups and the runbook

Companion documents: PRD, Module Breakdown, Engineering Design, Deployment Guide, Database Guide

1. What You Are Buying, and From Whom

You need one thing: **a small Linux server on the internet**, rented by the month. That is it. Everything else — the app, the database, the web server — runs inside that one machine as Docker containers.

That kind of rented machine is called a **VPS** (Virtual Private Server) or a "cloud instance". Several companies sell them, and they are direct competitors — you buy from **one** of them, not through AWS.

> **Revised September 2026.** This section originally recommended Hetzner Singapore at ≈₹400–600. Two things have since made that advice wrong, and both were found the hard way when the server was actually being bought:
>
> 1. **Hetzner's CX line is EU-only.** Singapore never had CX22 or CX23 — it sells the AMD `CPX` line instead. The old instruction to pick "CX22 in Singapore" described a machine that does not exist. 2. **Hetzner raised cloud prices sharply on 15 June 2026.** CX22 went from €7.99 to **€19.49/month** — a 144% rise. Comparable CPX increases were around 170–190%.
>
> The result is that Hetzner is no longer the cheap option. It is now the most expensive of the realistic choices *and* the furthest away.

1.1 What things actually cost now

Verify each of these yourself before paying — as the box above shows, they move.

Provider	Nearest region	Specs	Per month	Latency from Kerala
AWS Lightsail (chosen)	**Mumbai**	2 vCPU / 2 GB / 60 GB / 3 TB	$12 ≈ ₹1,060 + 18% GST	~20–30 ms
Vultr	Mumbai	1 vCPU / 2 GB / 55 GB / 2 TB	$10 ≈ ₹880 + 18% GST	~20–30 ms
DigitalOcean	Bangalore	1 vCPU / 2 GB / 50 GB / 2 TB	$12 ≈ ₹1,060 + 18% GST	~20–30 ms
Hetzner	Helsinki (EU)	2 vCPU / 4 GB / 40 GB	€19.49 ≈ ₹2,000, no GST	~150 ms

Decision: AWS Lightsail, Mumbai. An India region either way — 20–30 ms from Kerala against roughly 150 ms for Europe, and the shop's sales records stay in India, which is one fewer thing to explain to a CA. Lightsail bills a flat monthly figure with a generous transfer allowance rather than metering every component the way EC2 does, and it is the same AWS account if anything else is ever needed.

The one thing to know before you buy. Lightsail has **no in-place resize**. Growing means taking a snapshot, creating a larger instance from it and moving the static IP across (§8b) — about twenty minutes, done after the shop closes — and you **cannot go back down** to a smaller plan afterwards. Attach a static IP on day one, or the rebuilt instance comes up on a different address and DNS breaks. Vultr and DigitalOcean resize a running machine in place at similar prices; they are the alternatives if that rule matters more than staying inside AWS.

Nothing else in this guide is Lightsail-specific. Every provider gives you the same thing: an Ubuntu machine and an IP address. §3 onwards applies unchanged.

1.2 You do not need production yet

Right now you need **staging** — somewhere the Alpha can be shown. Production does not exist until go-live (M14). So buy the smaller machine now and size production properly later, when you know the real load:

- **Staging today:** the smallest **1 GB** Lightsail instance (~$5–6 ≈ ₹500 + GST). The server never compiles anything — it pulls a pre-built image — so 1 GB is enough for Postgres, the app and Caddy with room to spare.

- **Production at go-live: 2 GB**, not 1 GB. Node holds 200–400 MB under load, Postgres wants its buffers and working memory, and the report and analytics queries are the memory-hungry part. 1 GB *runs* a small shop, but with almost no headroom — and because Lightsail will not let you shrink later, the safe order is small for staging and right-sized for production. Revisit the price then; any figure written here today will have moved.

1.3 Total monthly cost

For **staging only**, which is where the project is now:

Item	₹ / month
1 GB Lightsail instance, Mumbai	~₹500 + GST
Lightsail automated snapshots (+~20%)	~₹100
Cloudflare R2 — file storage + off-site backups (free tier, 10 GB)	₹0
Resend — transactional email (free tier, 3,000/month)	₹0
Sentry — error tracking (free tier)	₹0
Domain name (`.in`, ~₹800/year)	~₹70
Total	**≈ ₹700–800**

Add roughly ₹**500–600** more when production comes up on a 2 GB instance alongside it. The original ₹550–750 estimate for a *production* box no longer holds: the honest figure today is ₹**1,200–1,400 all-in** once both are running. That is the market moving, not a change of plan — and it is still an order of magnitude below what a managed platform would charge.

1.4 A 2 GB box needs Postgres told to be modest

The defaults assume a machine with far more memory. On a 1–2 GB instance, add this to the `db` service in `docker-compose.prod.yml`:

```
    command: >
      postgres
      -c shared_buffers=256MB
      -c effective_cache_size=768MB
      -c work_mem=8MB
      -c maintenance_work_mem=64MB
      -c max_connections=50
```

Without it Postgres will happily reserve more than the box has and the kernel will kill something — usually the app, at the worst possible moment.

2. Day One: The Shopping List

Do these four things before touching any code. Budget about an hour.

2.1 Create an account with your chosen provider (see §1.1 — Vultr Mumbai or DigitalOcean Bangalore). You will need a credit/debit card. New accounts are sometimes asked for an identity document before the first server can be created; this is normal and usually clears within a few hours. Start this first so the wait does not block you.

If you already opened a Hetzner account: keep it, it costs nothing dormant. Their Singapore region does not sell the machine this guide assumed, and their EU prices tripled in June 2026 — see §1.1.

2.2 Buy a domain name. Any registrar works — Namecheap, Cloudflare Registrar, or an Indian registrar like BigRock. A `.in` domain runs about ₹800/year. You will point it at the server in §5.

2.3 Create a Cloudflare account (free) — `cloudflare.com`. You need it for two things: free DNS management, and **R2** object storage for backups and uploaded bill photos. R2's free tier is 10 GB of storage with zero egress fees, which will cover this shop for a long time.

2.4 Create a GitHub account and a private repository for the code, if you have not already. Deployments will run from here.

---

3. Creating the Server

3.1 Make an SSH key first

An SSH key is how you log in to the server — safer than a password, and you never type it. On your Mac, in Terminal:

```
ssh-keygen -t ed25519 -C "ecity-server"
# press Enter to accept the default path
# set a passphrase when asked, and remember it

cat ~/.ssh/id_ed25519.pub     # this prints the PUBLIC key - copy it
```

The `.pub` file is the **public** key and is safe to paste anywhere. The file without `.pub` is your **private** key — it never leaves your Mac and is never shared, committed or emailed.

3.2 Create the server — AWS Lightsail, step by step

Lightsail is deliberately not the EC2 console: one page, a flat price, and no VPC to configure. From `lightsail.aws.amazon.com`:

1. Create instance

Setting	Choose	Why
Region	**Mumbai (ap-south-1)**	20–30 ms from Kerala, and the records stay in India
Platform	**Linux/Unix**	
Blueprint	**OS Only** → **Ubuntu 24.04 LTS**	Not an app blueprint — everything runs in Docker
Plan	**$5 (1 GB)** for staging, **$12 (2 GB)** for production	§1.2. Lightsail cannot shrink later, so size production properly now
SSH key	**Upload the public key from §3.1**	Lightsail offers to generate one; use your own, so the private key never touches AWS
Name	`ecity-staging` or `ecity-prod`	

Choose the region **before** anything else — it cannot be changed afterwards, and a static IP only attaches to an instance in the same region.

2. Attach a static IP — do this immediately

Networking → **Create static IP** → attach it to the instance.

It is free while attached, and it is what makes §8b's upgrade possible: when you outgrow the plan you rebuild a larger instance and move this IP across, and DNS never notices. Skip it and the address changes on every rebuild. (Detached static IPs are billed, so release one you stop using.)

3. Firewall — Networking → IPv4 Firewall. Delete anything not listed:

Application	Port	Source	Why
SSH	22	**your IP** if it is static, else Any	Your login
HTTP	80	Any	Redirect to HTTPS, and Let's Encrypt renewals
HTTPS	443	Any	Everything real

Lightsail opens **3306 and 5432** on some blueprints. Delete them if present. Postgres is never exposed: the app reaches it over Docker's internal network, and an open Postgres is found by scanners within hours.

4. Snapshots — Snapshots tab → **Enable automatic snapshots**, and pick an hour the shop is closed. This is backup layer 1 (§7); roughly 20% of the instance price, and not the layer to save money on.

5. Note the IP. You now have a static IPv4 like `13.234.x.x`. It goes in the DNS record in §5.

On other providers — Vultr, DigitalOcean, Linode — the same choices exist under slightly different labels: Mumbai or Bangalore, Ubuntu 24.04, 1 or 2 GB, your SSH key, a firewall allowing only 22/80/443, and automated backups on. Everything from §3.3 onwards is identical.

3.3 First login and hardening

```
ssh root@YOUR_SERVER_IP

# 1. create a normal user for day-to-day work
adduser ecity
usermod -aG sudo ecity
rsync --archive --chown=ecity:ecity ~/.ssh /home/ecity

# 2. updates and basic protection
apt update && apt upgrade -y
apt install -y ufw fail2ban unattended-upgrades
dpkg-reconfigure --priority=low unattended-upgrades

# 3. host firewall (second layer behind the provider's own)
ufw allow OpenSSH
ufw allow 80/tcp
ufw allow 443/tcp
ufw --force enable

# 4. turn off password logins entirely
sed -i 's/^#*PermitRootLogin.*/PermitRootLogin prohibit-password/' \
    /etc/ssh/sshd_config
sed -i 's/^#*PasswordAuthentication.*/PasswordAuthentication no/' \
    /etc/ssh/sshd_config
systemctl restart ssh
```

Now open a **second** terminal and confirm `ssh ecity@YOUR_SERVER_IP` works before closing the first one. If you lock yourself out, your provider's web console gets you back in — but check first anyway.

3.4 Install Docker

```
curl -fsSL https://get.docker.com | sh
usermod -aG docker ecity
# log out and back in as ecity, then verify:
docker run --rm hello-world
```

3b. Cloudflare R2 — buckets and keys

Two buckets, one credential pair. Do this before the first deploy: the app reads these at startup, and the backup script refuses to run without them.

1. Turn R2 on. Cloudflare dashboard → **R2** → *Enable*. It asks for a card even on the free tier; 10 GB of storage and zero egress costs nothing, and this shop will not approach that for years.

2. Create two buckets, both in an automatic or Asia-Pacific location:

Bucket	Holds	Who writes it
`ecity-uploads`	Bill photos and attachments people add in the app	The application
`ecity-backups`	Nightly database dumps, via restic	`scripts/backup.sh` on the server

Separate on purpose. The uploads bucket is written by the app on every attachment; the backup bucket is the thing you need on the worst day, and it should not share a lifecycle, a retention rule or an accident with anything else.

3. Create an API token. R2 → **Manage R2 API Tokens** → *Create token*:

- Permission: **Object Read & Write**
- Scope: **the two buckets above**, not "all buckets"
- TTL: no expiry (or diarise the renewal — an expired token means backups stop silently, which is the failure this whole section exists to avoid)

It shows you three values **once**:

```
R2_ACCOUNT_ID=<the long hex id, also in the R2 endpoint URL>
R2_ACCESS_KEY_ID=<access key>
R2_SECRET_ACCESS_KEY=<secret — shown only now>
```

Put them straight into `.env` on the server (§4.3). If you lose the secret you cannot recover it; you roll the token and update `.env`.

4. Add a restic password. The backup repository is encrypted, and this is the key:

```
openssl rand -base64 32     # RESTIC_PASSWORD
```

Store it somewhere that is not the server. A password manager, or written down at home. If the server dies and this is only on the server, the off-site backups are unreadable — encrypted rubbish. That is the single most common way a backup strategy turns out to be theatre.

5. Turn backups on — one command:

```
cd ~/app && ./scripts/setup-backups.sh
```

It installs restic, initialises the encrypted repository, schedules the job every four hours, and then **takes a real backup and shows you the snapshot**. That last step is the point: the four steps used to be manual, and the one people skip is the proof — leaving a cron entry that has never succeeded, which looks exactly like one that has.

Safe to run again; every step checks before it acts, so a second run repairs whatever is missing rather than duplicating it.

`RESTIC_REPOSITORY` does **not** belong in `.env` — the scripts build it from `R2_ACCOUNT_ID` and `R2_BACKUP_BUCKET`.

3b.1 Wiring uploads to R2

Attachments — bill photos, supplier invoices, product images — are stored in the `ecity-uploads` bucket. Four variables in `.env`:

```
STORAGE_DRIVER=s3
R2_ACCOUNT_ID=<the long hex id from the R2 dashboard>
S3_BUCKET=ecity-uploads
S3_ACCESS_KEY_ID=<from the API token>
S3_SECRET_ACCESS_KEY=<from the API token>
```

`S3_ENDPOINT` can replace `R2_ACCOUNT_ID` if you ever move to another provider — "S3" here is the *protocol*, not Amazon. R2 speaks it, and so do Backblaze B2 and MinIO.

Leave `STORAGE_DRIVER` unset and uploads go to the container's own disk, where the next deploy destroys them — the container is replaced, and the files are not in the database dump either. That is fine on a laptop and wrong on a server, which is why the driver refuses to start with a half-filled configuration: if `STORAGE_DRIVER=s3` and a variable is missing, the first upload fails naming the variable, rather than quietly writing to a disk that is about to disappear.

Uploads are capped at 5 MB, which fits a phone photo with room to spare and keeps a shop inside R2's free 10 GB — roughly two thousand photos. Anything larger is refused from the `content-length` header before the file is read, so a large upload cannot exhaust the memory of a 1 GB instance.

To check it works, attach a photo to a purchase, then confirm the object appears in the bucket:

```
# in the Cloudflare dashboard: R2 -> ecity-uploads -> Objects
```

Then deploy again and confirm the photo is **still there** — that is the whole point, and the one thing the local driver cannot do.

4. The Application Stack

Everything lives in `/home/ecity/app` on the server. Four containers:

```
  Internet :443
      |
   [caddy]      TLS certificates, automatic and free
      |
   [app]        Next.js (long-running Node process, port 3000)
      |
   [db]         PostgreSQL 16          [worker]  pg-boss jobs
```

Because the app is a **long-running process**, not serverless functions, there is no cold start and no database connection-pool problem. One small pool of ~10 connections is opened once and reused.

4.1 `docker-compose.yml`

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: ecity
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ecity
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ecity"]
      interval: 10s
      retries: 5

  app:
    image: ghcr.io/YOURNAME/ecity:latest
    restart: unless-stopped
    env_file: .env
    depends_on:
      db: { condition: service_healthy }
    expose: ["3000"]

  worker:
    image: ghcr.io/YOURNAME/ecity:latest
    command: ["node", "dist/jobs/worker.js"]
    restart: unless-stopped
    env_file: .env
    depends_on:
      db: { condition: service_healthy }

  caddy:
    image: caddy:2-alpine
    restart: unless-stopped
    ports: ["80:80", "443:443"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    depends_on: [app]

volumes:
  db_data:
  caddy_data:
  caddy_config:
```

4.2 `Caddyfile`

Two lines gets you HTTPS with a free certificate that renews itself:

```
app.yourshop.in {
    encode gzip
    reverse_proxy app:3000
}
```

Caddy places no limit on request body size, which is what the go-live import needs — a file goes up in one request, and a spreadsheet is sent as bytes, so a shop's stock list can be several megabytes. If you ever add a `request_body max_size` here, keep it above 15 MB or the import will fail at the one moment it matters. (Nginx is the other way round: its default of 1 MB would need raising.)

4.3 `.env` on the server

Create it with `nano .env`, then `chmod 600 .env`. **This file is never committed to git.**

```
POSTGRES_PASSWORD=<long random string>
DATABASE_URL=postgres://ecity:<same password>@db:5432/ecity
```

```
AUTH_SECRET=<openssl rand -base64 32>
AUTH_URL=https://app.yourshop.in
R2_ACCOUNT_ID=...
R2_ACCESS_KEY_ID=...
R2_SECRET_ACCESS_KEY=...
R2_BUCKET=ecity-uploads
RESEND_API_KEY=...
SENTRY_DSN=...
NODE_ENV=production
```

Generate secrets with `openssl rand -base64 32`. Never reuse the same value twice.

4.4 `Dockerfile` (in the repo)

Next.js needs `output: 'standalone'` in `next.config.ts` for this to work — set that first.

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci

FROM node:22-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:22-alpine AS run
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./.next/static
COPY --from=build /app/public ./public
EXPOSE 3000
CMD ["node", "server.js"]
```

5. Domain and DNS

1. In Cloudflare, add your domain and change the nameservers at your registrar to the two Cloudflare gives you. Propagation takes minutes to a few hours.
2. Add one DNS record:

Type	Name	Value	Proxy
A	`app`	your server IP	**DNS only (grey cloud)**

Keep the proxy **off** at first so Caddy can obtain its certificate.

3. Once DNS resolves, `docker compose up -d` and Caddy fetches a Let's Encrypt certificate automatically. `https://app.yourshop.in` is live.

5.1 Cloudflare does not replace Caddy

A common and expensive misunderstanding, so it is worth being explicit.

Cloudflare terminates TLS at **its** edge — between the browser and Cloudflare. It does nothing about the leg between Cloudflare and your server, and that leg crosses the public internet. Something on the machine has to terminate TLS on it, and that is Caddy. Caddy is also the reverse proxy that puts requests onto the app container; Next.js cannot listen on 443 itself. Remove Caddy and nothing answers the port.

Cloudflare's SSL modes make the point:

Mode	Browser → Cloudflare	Cloudflare → your server	Use it?
Flexible	encrypted	**plain HTTP**	**Never.** Bills and customer data in clear across the internet, and it causes redirect loops
Full	encrypted	encrypted, certificate unchecked	Still needs a certificate on the box
Full (strict)	encrypted	encrypted and verified	**This one**, if you proxy at all
Off	—	—	No

Every usable mode needs a certificate on the server. Turning the proxy on changes only where Caddy's certificate comes from — never whether Caddy is there.

5.2 If you turn the orange cloud on

You do not have to. Caddy plus Let's Encrypt is two lines of config and renews itself; the proxy buys DDoS protection and caching, neither of which a single shop needs for correctness. If you do want it, three things have to be right and the third is the one that fails silently.

1. Set SSL/TLS mode to Full (strict). Then give Caddy a certificate Cloudflare will accept: either keep Let's Encrypt (port 80 must stay reachable for the HTTP challenge, or switch Caddy to the DNS challenge with a Cloudflare API token), or install a free **Cloudflare Origin Certificate** — fifteen-year validity, trusted only by Cloudflare, which is exactly this job.

2. Restrict the firewall to Cloudflare's IP ranges. Otherwise anyone who discovers the origin IP connects to it directly, bypassing Cloudflare completely, and you have paid for nothing.

3. The rate limiter depends on step 2. Sign-in throttling identifies the caller from `x-forwarded-for` (M14). Cloudflare sets the real client address as the leftmost entry, so it keeps working when proxied — but on an origin that still accepts direct connections, anyone can send whatever `x-forwarded-for` they like and the per-address limit becomes trivial to evade. Locking the firewall to Cloudflare is what makes the header trustworthy. The two are one change, not two.

6. Deploying

Build the image in GitHub Actions, push it to GitHub's container registry, then have the server pull it. The server never compiles anything — it only downloads and restarts, so a deploy takes seconds and a bad build never reaches production.

6.1 The workflow

The real file is `.github/workflows/deploy.yml` in the repository — read that rather than a copy here, since a copy in a document goes stale. What it does, and *why* each part is the way it is:

It refuses to deploy code that fails. The deploy job declares `needs: [check]`, and `check` calls the whole CI workflow. A push whose tests, types or lint fail never reaches the server. An earlier version of this file had `needs: []`, which meant a red build shipped anyway — worse than no automation at all, because it looks safe.

Two images are built, not one.

Tag	What it is	Why
`:latest`, `:<sha>`	the Next.js standalone server	what serves customers
`:migrate`	node_modules + `drizzle/` + the db scripts	runs migrations, then exits

The runtime image is a `.next/standalone` bundle. It has **no** `drizzle-kit` **and no** `tsx` — those are dev dependencies, and shipping a toolchain into the container that faces the internet is not worth it. So migrations get their own small image which is run once per deploy and thrown away. (The original workflow tried to run `drizzle-kit` from the app image with `|| true` on the end. It could never have worked, and the `|| true` would have hidden that forever while the app served traffic against an un-migrated schema.)

The order matters, and nothing is allowed to fail quietly:

```
docker compose pull
docker compose run --rm migrate                    # schema first
docker compose run --rm migrate npm run db:sync-roles   # then permissions
docker compose up -d --wait app caddy              # then release
curl /api/health                                   # then prove it
```

`script_stops: true` plus `set -euo pipefail` means any step failing stops the deploy with a red build, instead of carrying on to start a broken release.

Why `db:sync-roles` **is a deploy step.** A module that adds a permission adds it to `SYSTEM_ROLES` in the code. Roles already in the database keep whatever they were seeded with, so the new screen returns 403 for *everyone* — including the owner — until the roles are re-synced. M5 hit exactly this: `Customer dues` was built, deployed, and invisible. The sync is idempotent, touches only roles marked `is_system`, and leaves anything the shop created itself alone.

The health check is not optional. Ten one-second tries against `/api/health`; if none succeed the job prints the last 50 lines of the app log and fails. Without it a deploy that crash-loops reports success.

Add `SSH_HOST` (the server IP) and `SSH_KEY` (your **private** key) as GitHub repository secrets under Settings → Secrets and variables → Actions.

6.2 Standing up staging (M5)

There is a step-by-step walkthrough for Vultr Mumbai at `deploy/staging/README.md`, with the compose file, Caddyfile and `.env` template beside it, already filled in for this repository. Follow that. The summary below is what it does and why.

Everything above is written and committed. What is left needs **your provider account and card**, so it cannot be automated from here. Roughly 40 minutes:

1. **Create the server.** Follow §3 exactly, but name it `ecity-staging`. A The smallest 1 GB instance in an India region is enough for staging (§1.2) — about ₹500/month. Note the IP.
2. **Harden it.** §3.1 as written — normal user, firewall, no password logins.
3. **Put the stack on it.**

```
ssh ecity@<ip>
mkdir -p ~/app && cd ~/app
# copy docker-compose.prod.yml, Caddyfile and .env from the repo
# in docker-compose.prod.yml replace YOURNAME with your GitHub username
```

4. **Write** `.env` with a *staging* `DATABASE_URL`, a freshly generated `AUTH_SECRET` (`openssl rand -base64 32`), and `POSTGRES_PASSWORD`. Never reuse production values.
5. **Give it a hostname.** The session cookie is *Secure* in production mode, so a bare `http://<ip>` shows the login page and then silently refuses to log anyone in — HTTPS is not optional. Staging avoids buying a domain by using `<ip>.sslip.io`, a free public DNS service that resolves any such name to that IP; Caddy then gets a real certificate for it automatically. Swap in a proper domain whenever you buy one.

Use the **IPv4** address. `curl ifconfig.me` returns IPv6 on a dual-stack server, and an IPv6 address contains colons, which are illegal in a domain name — Let's Encrypt refuses to issue, and the site then hangs with nothing obvious in the browser. Force it with `curl -s -4 ifconfig.me`.

6. **Give GitHub the keys.** Repository → Settings → Secrets → Actions: `SSH_HOST` = the IP, `SSH_KEY` = the private key that matches the one you put on the server.
7. **Let the registry pull.** On the server, `docker login ghcr.io -u <github-username>` with a personal access token that has `read:packages`.
8. **Push to `main`.** The workflow builds, migrates, syncs roles, releases and health-checks. Watch it under the Actions tab.
9. **Seed it once**, so there is something to log in with:

```
docker compose run --rm migrate npm run db:seed
```

Staging only. The seed creates accounts with known passwords.

Verify it worked: open `https://staging.yourdomain.com`, sign in, and walk the M0 section of the Manual Test Checklist. If the health check passed but the site does not load, the problem is DNS or Caddy, not the app — check `docker compose logs caddy`.

6.3 Two rules about migrations

1. **Always take a backup before a migration that drops or renames anything.** `./backup.sh` first, every time.
2. **Test every migration on staging before production.** Staging can be a second, smaller instance (~₹500) or just a second Docker stack on the same box using a different database name and port. Cheap either way, and it is what stops a bad migration reaching real sales data.

---

7. Backups — The Part You Cannot Skip

You are running your own database now, so backups are entirely your responsibility. This is the only real thing you gave up by not paying for managed hosting. Three layers:

Layer	What it protects against	Frequency
Provider snapshots	Server dies, disk corrupts	Daily, automatic
`pg_dump` to Cloudflare R2	Bad migration, wrong DELETE, provider outage	Nightly, off-site
Restore drill	Backups that were never actually valid	**Monthly, by hand**

7.1 `scripts/backup.sh`

It ships with the app — `scripts/backup.sh` in the repository, so it is version-controlled and reviewed like anything else rather than pasted onto a server once and forgotten.

It does three things a hand-written dump does not, each of which has cost somebody their data somewhere:

- **It fails loudly.** A backup that exits 0 after a failed dump buys a year of false confidence and is discovered on the day it is needed.
- **It dumps with `-Z 0`** so restic can deduplicate between snapshots. Compressed dumps differ completely each night, and seventeen retained snapshots would then cost seventeen full copies — which is what keeps the history inside R2's free 10 GB.
- **It checks what it produced.** A dump under a size floor is refused, and the archive's table of contents is read back with `pg_restore --list` before it is called a backup. An empty dump restores perfectly into an empty

database; silence is the failure mode worth designing against.

```
# on the server, where the database is a container
cd /home/ecity/app && ./scripts/backup.sh

# anywhere with a connection string, no restic
DATABASE_URL=... ./scripts/backup.sh --local
```

With `-Z 0` and the retention below, seventeen retained snapshots deduplicate down to roughly the size of one or two — which is what keeps the whole backup history inside Cloudflare R2's free 10 GB.

Install with `apt install -y restic`, initialise once with `restic init`, then `chmod +x backup.sh` and schedule it — `crontab -e`:

```
# every four hours, so a crash costs at most one quarter of a trading day
30 */4 * * * /home/ecity/app/backup.sh >> /var/log/ecity-backup.log 2>&1
```

A nightly-only schedule means a disk failure at 8 PM loses **everything entered that day** across every branch. Four-hourly costs nothing extra and cuts the worst case to about four hours. For true point-in-time recovery — restoring to any moment, including just before a bad migration — add WAL archiving with `pgBackRest` or `wal-g` shipping to the same R2 bucket. That is roughly half a day of setup in M13 and brings the worst case down to about five minutes.

7.2 The monthly restore drill

Put a recurring reminder in your calendar. It takes ten minutes, and it is also a script — `scripts/restore-drill.sh` — because a drill somebody has to remember the steps for is a drill that gets skipped.

```
# newest local dump, or name one
restic restore latest --target /tmp/restore     # only if pulling from R2
./scripts/restore-drill.sh
```

It restores into a **scratch** database (never over the live one — finding out a backup is bad on top of real data is the disaster), then reports:

- how many sales, devices, ledger entries and cash movements came back;
- whether the **append-only guards** came back *and still refuse a write* — it attempts a forbidden update and expects to be rejected. A restored database that is writable in ways the live one never was is the copy somebody would promote on a bad day;
- **how long the restore took.** Write that number down: it is how long the shop is shut for.

If the counts look right, your backups work. If you have never done this, **you do not have backups** — you have files you hope are backups.

---

8. Monitoring

What	Tool	Cost
Application errors	Sentry (free Developer tier)	₹0
Site down alert	UptimeRobot, 5-minute checks to email/SMS	₹0
Disk full	`df -h` in a weekly cron that emails if above 80%	₹0
Backup failed	Have `backup.sh` email or Sentry-alert on non-zero exit	₹0

Disk filling up is the single most common way a small VPS falls over. Old Docker images are usually the cause — `docker image prune -f` runs on every deploy in the workflow above, which handles it.

8b. Upgrading the server to a bigger plan

Do this **after the shop closes**. Not for the reboot — for the twenty minutes between taking the snapshot and switching over, during which anything billed would be written to the old machine and then thrown away. Closing time removes that risk completely.

On Lightsail there is no in-place resize. You snapshot, create a larger instance from the snapshot, and move the static IP across. DigitalOcean and Vultr can resize a running instance in place; Lightsail cannot, and it will not let you go back *down* to a smaller plan afterwards. Attach a static IP on day one, or the new instance comes up on a different address and DNS breaks.

The snapshot is the whole disk — Docker, the compose file, `.env`, the Caddy config and the database volume. Nothing is reinstalled and no deploy is needed; the application does not know it moved.

1. Record what should be there, and take a dump as a fallback

```
cd ~/app
docker compose exec -T db pg_dump -U ecity -Fc ecity > ~/pre-upgrade.dump
docker compose exec -T db psql -U ecity -d ecity -c \
  "select (select count(*) from sale) as sales,
          (select count(*) from device_unit) as devices,
          (select count(*) from customer_ledger_entry) as ledger;"
```

Write those three numbers down.

2. Stop everything, so the snapshot catches a closed database

```
docker compose down
```

A snapshot of a *running* instance is crash-consistent — the equivalent of pulling the power cord. Postgres replays its write-ahead log and usually recovers, but "usually" is the wrong standard for a sales ledger. Stopped, it flushes and closes properly.

3. Snapshot, create the larger instance, move the static IP

All three in the provider console.

4. Start it and check the numbers match

```
cd ~/app && docker compose up -d
docker compose exec -T db psql -U ecity -d ecity -c \
  "select (select count(*) from sale) as sales,
          (select count(*) from device_unit) as devices,
          (select count(*) from customer_ledger_entry) as ledger;"
curl -s https://YOUR_HOST/api/health
```

The three counts must match step 1 **exactly**, and health must report `"schema":"current"`.

5. Only then delete the old instance

Until you do, you have two ways back: the old machine, still intact, and `pre-upgrade.dump`. Keep both until the shop has traded a full day on the new one.

8c. Emptying staging to start testing

Wiping staging back to an empty shop, keeping only a login. Everything runs through Docker, because the server has no Node and no `psql` — only the containers (§4).

Order matters: deploy first. `db:seed` runs from the *migrate image on the server*, and that image only changes when a deploy pushes a new one. Reset before deploying and you reseed from whatever the server last pulled — which may still create demo branches and tax rates you have to delete again. Push, wait for the green tick in GitHub Actions, and only then do this.

1. Get on the server

```
ssh ecity@<staging-ip>
cd ~/app
```

The user is `ecity`, not `ubuntu` — §3.3 creates it. `~/app` is where `docker-compose.yml` and `.env` live.

2. Back up first

```
docker compose exec -T db pg_dump -U ecity -Fc ecity > ~/before-reset.dump
```

`exec -T db` runs inside the Postgres container; the host has no `pg_dump`. `-Fc` is the compressed format `pg_restore` reads.

Then confirm it is real, not an error message in a file:

```
ls -lh ~/before-reset.dump                                  # MB, or at least hundreds of KB
docker compose exec -T db pg_restore --list < ~/before-reset.dump | grep -c "TABLE DATA"
```

The second should print roughly the number of tables in the schema (about 57). A few hundred **bytes** means the dump failed.

This is a safety net for the next ten minutes, not a backup — it sits on the same disk as the database it came from. Real backups are §7.

3. Stop the app

```
docker compose stop app
docker compose ps
```

The app holds a pool of open connections, and a database cannot be dropped while anything is attached. `db` and `caddy` stay running — you need Postgres up to reset it.

Staging has no `worker` *service* (§4.1 defines `db`, `app`, `migrate` and `caddy`). If you add one later, stop it here too.

4. Drop and recreate

```
docker compose exec -T db psql -U ecity -d postgres -c "drop database if exists ecity with (force);"
docker compose exec -T db psql -U ecity -d postgres -c "create database ecity;"
```

Connect to `postgres`, the maintenance database — you cannot drop the one you are connected to. `with (force)` disconnects anything that reconnected after step 3.

5. Rebuild the schema

```
docker compose run --rm migrate
```

Every migration in order, from nothing to current. The migration tooling is a separate image; the app image does not carry it. **Wait for** `Migrations applied.` — seeding a half-built schema fails confusingly.

If it says there is no such service, add the profile the service sits behind:
`docker compose --profile tools run --rm migrate`.

6. Seed the login

```
docker compose run --rm migrate npm run db:seed
```

Creates the permission catalogue, the three system roles, the business and **one admin**. No branches, tax rates, payment methods or catalogue — those belong to the shop.

Not `db:seed:demo`**.** That adds two demo branches, GST rates, a fake catalogue and extra logins. It exists so the browser suite has something to bill against, and has no place on a server a shop will use.

Note the password it prints; it comes from `SEED_PASSWORD` in `.env`, or falls back to `ChangeMe!2026`.

7. Start it again and check

```
docker compose start app && docker compose ps
docker compose exec -T app node -e "fetch('http://localhost:3000/api/health').then(r=>r.json()).then(d
=>console.log(JSON.stringify(d)))"
```

Asked from *inside* the container: the app publishes port 3000 only on the Docker network, so `curl` on the host never connects. The health check reports whether migrations are current — a half-applied migration looks exactly like a broken feature once you start clicking.

8. Set the shop up, in this order

Sign in at `https://<ip>.sslip.io` as `admin@ecity.local`. It forces a password change on first use — that is the first test.

Order	Where	Why it is first
1	**Settings → Branches**	Nothing can be billed without one. Billing offers an "Add the first branch" button if you forget
2	**Settings → Business**	Shop name, the GST toggle, tax rates, and **at least one payment method** — a sale cannot be paid for without one
3	**Settings → Catalogue**	Brands and categories. A category decides IMEI-tracked or counted, and that **locks once products exist**
4	**Settings → Users**	The real people

Then a purchase to bring stock in, then a bill.

Two things to expect. Bill photos disappear on the next deploy until uploads are finished (§3b.1). And alerts only appear when the worker evaluates them — with no worker on staging, use **Check now** on the Alerts page.

9. Runbook

Common operations, for when you need them and cannot remember.

```
# where everything lives
cd /home/ecity/app

# see what is running / logs
docker compose ps
docker compose logs -f app
docker compose logs --tail=200 worker

# restart one service
docker compose restart app

# deploy by hand (normally GitHub Actions does this)
docker compose pull && docker compose up -d

# roll back to the previous image
docker compose pull ghcr.io/YOURNAME/ecity:<previous-sha>
# edit docker-compose.yml to that tag, then:
docker compose up -d

# open a database shell
```

```
docker compose exec db psql -U ecity ecity

# take a backup right now
./backup.sh

# disk and memory
df -h && free -m && docker system df
```

Resizing the server (when 10 branches actually arrive): provider console → Server → Resize → the next tier up (4 vCPU / 8 GB). It reboots once, takes about a minute, and nothing else changes. You can scale up freely; scaling disk down is not possible on any of these providers, so do not over-provision the disk early.

10. Loading the Shop's Existing Data (PRD OQ-9)

This is the answer to OQ-9. The importer is mapping-driven, so it never needed to know the shop's file layout in advance — the question that remained was *which files, in what order, and how do we know it worked*. That is a plan, and this is it.

The rule that sets the order: every import runs through the same services as manual entry, so a row can only reference something that already exists. A device needs its product; a due needs the customer it belongs to. Load in this order and nothing is ever waiting on something that has not arrived yet.

Before any file: set the shop up by hand

These are minutes of typing, not an import, and everything else hangs off them.

- [ ] Business details, GST toggle, financial year, invoice prefix
- [ ] **Branches** — codes matter; they appear on every document number
- [ ] Tax rates and payment methods
- [ ] Roles, then users, each assigned to their branches
- [ ] Bank / UPI / card **accounts**

Then the files, in this order

#	What	Needs	Where	Notes
1	**Categories and brands**	—	Settings → Catalogue	Optional: the product import creates a missing category as a *counted* one. Create anything IMEI-tracked here **first**, because that setting locks once products use it
2	**Products**	categories	Import → Products	`name` and `category` required. Bring `sku`, `purchasePrice`, `sellingPrice` if the shop has them
3	**Customers**	—	Import → Customers	`name` required; `phone` is what the counter searches by, so include it
4	**Suppliers**	—	Import → Suppliers	As above
5	**Handsets in stock**	products, branches	Import → Handsets (by IMEI)	Choose the branch on the upload. One row per handset, with `mainType`; several IMEI columns per row are read. Include `receivedAt` if known, or every handset dates to the day you uploaded
6	**Opening stock (accessories)**	products, branches	Opening balances → Stock, or Import → Opening stock	Counts as they stand on the day you start
7	**Opening cash and account balances**	branches, accounts	Opening balances → Cash & accounts	Once per branch — a second figure is refused
8	**Customer dues**	customers	Opening balances → Dues, or Import → Opening customer dues	A name the shop does not have stops that row rather than inventing an account
9	**Supplier dues**	suppliers	Opening balances → Dues, or Import → Opening supplier dues	As above

Steps 6–9 are typed in when there are a handful and filed when there are hundreds; both go through the same services, so the result is identical.

What to ask the shop for

One question per file, in the shop's own words:

- *"A list of everything you sell"* → products
- *"Every phone on the shelf, with its IMEI"* → handsets. **The single most important file**, and the one most likely to be on paper
- *"Everyone who owes you money, and how much"* → customer dues
- *"Everyone you owe, and how much"* → supplier dues
- *"How much is in the till and the bank today"* → opening cash
- *"How many of each accessory are on the shelf"* → opening stock

CSV or `.xlsx`, first sheet, one heading row. Anything else — a photo of a register, a WhatsApp list — becomes a spreadsheet first. **Nothing is created until the preview has been looked at**, so an ugly file is safe to try.

Checks after each step, before the next

- [ ] The batch says how many rows imported and how many had problems
- [ ] **Download the problem rows**, fix them in the original file, and re-upload *just those* — the same whole file is refused by content hash

- [ ] Spot-check five records against the source by hand
- [ ] After handsets: the device count matches the file, and a known IMEI is findable in the search box
- [ ] After dues: the total on the dues screen matches the shop's own figure. *If it does not, stop — everything after this is built on it*

Two things to decide with the shop before starting

1. **The cut-off date.** Every opening figure is "as at" one day. Trading on both systems either side of it is what makes a parallel run reconcilable.
2. **Who checks.** The person who knows what the numbers *should* be has to confirm each step, and it should not be whoever ran the import.

What is deliberately not migrated

Past sales, purchases and payments. ECITY starts from balances, not from history: a re-keyed invoice is not the invoice that was issued, and a half-migrated sales history is worse than none — the reports would look complete while being wrong. The paper stays the record for anything before the cut-off, and the shop keeps its old files.

---

11. Go-Live Checklist

- [ ] Provider account verified, instance running in an India region, snapshots enabled
- [ ] Provider firewall: only 22, 80, 443 inbound. Port 5432 closed
- [ ] SSH key login working; password login disabled; `ufw` enabled
- [ ] Domain pointing at the server; HTTPS live with a valid certificate
- [ ] `.env` present, `chmod 600`, and **not** in git
- [ ] All secrets freshly generated; none reused from development
- [ ] GitHub Actions deploys on push to `main`; tests gate the deploy
- [ ] Staging environment exists and migrations are tested there first
- [ ] `backup.sh` scheduled nightly and confirmed writing to R2
- [ ] **A restore drill completed successfully and timed at least once**
- [ ] Sentry receiving errors; UptimeRobot alerting to a phone
- [ ] Existing data loaded and checked, in the order in §10
- [ ] Opening balances loaded (PRD FR-34), and the dues totals agreed with the shop
- [ ] Two-week parallel run with paper records agreed with the shop

---

12. Things That Will Bite You

Mistake	What happens	Prevention
Never testing a restore	You discover the backups were empty on the day you need them	The monthly drill in §7.2
Exposing Postgres port 5432	The database is found and attacked within hours	Never publish port 5432; Docker's internal network only
Secrets committed to git	Credentials leak permanently, even after deletion	`.env` in `.gitignore`; GitHub secrets for CI
Running migrations straight on production	One bad migration, no way back	Staging first, backup immediately before
Disk quietly filling up	Postgres stops accepting writes; the shop stops billing	`docker image prune` on deploy; weekly disk check
Editing files on the server by hand	The next deploy silently overwrites your fix	All changes go through git and CI, always
Only one person holds the SSH key	Nobody can reach the server if that laptop dies	Second key stored securely offline, or provider console access